CSE513 Assignment 1: CPU Scheduling Simulator

Website: https://meliksahcaliskan.github.io/Scheduler_Simulation/

Meliksah Caliskan · October 25, 2026

1 Overview and how to run

The simulator is a single web page that schedules a set of processes with FCFS, SJF (non-preemptive), SRTF (preemptive) or Round Robin with a configurable quantum. It shows a labeled Gantt chart (including idle intervals and, for RR, every quantum slice as its own block), a results table with all seven per-process values and the three averages, a verification panel that checks the five required properties on every run, a step-by-step player that explains each scheduling decision, and a comparison of all four algorithms on the same input.

- **Online:** `https://meliksahcaliskan.github.io/Scheduler_Simulation/` (GitHub Pages, no login needed).
- **Locally:** open `index.html` from the source archive by double-clicking it. It is one self-contained file (all JavaScript and CSS inline), so it needs no server, installation or network.
- **Source:** `https://github.com/Meliksahcaliskan/Scheduler_Simulation`. The `README.md` explains how to run, test and rebuild the page, the dependencies (none at runtime; Node.js only for the tests and the build) and the limitations.

Model and rules. One CPU; each process has one CPU burst and no I/O; context switches cost nothing; time is in integer units and intervals are half-open ($[2, 5)$ is 3 units). The CPU never idles while a process is ready. FCFS picks the earliest arrival; SJF picks the shortest burst when the CPU becomes free; SRTF picks the shortest remaining time and preempts only on a *strictly* shorter remaining time. Ties go to the earlier arrival, then the earlier row in the input. RR uses a FIFO queue; at a quantum boundary, processes arriving at that exact time are enqueued before the preempted process, and a finished process never rejoins the queue. Invalid input (empty or duplicate IDs, where `p1` and `P1` count as duplicates; non-integer or negative times; zero bursts or quantum; values over the limits) is rejected with one message per problem, tied to its row and marked on the field.

Implementation. The engine (`src/scheduler.js`) is a pure function that steps through time one unit at a time. At every integer time t it performs the same steps in a fixed order: (1) complete the process that ran in $[t-1, t)$; (2) admit arrivals at t in input order; (3) RR: expire the quantum and re-enqueue the process *after* the arrivals of step 2; (4) SRTF: preempt only if an arrival is strictly shorter; (5) dispatch or start an idle interval; (6) run one unit. The RR boundary rule and "finishing exactly at a boundary is a completion, not an expiry" follow from this order. Every decision is recorded as an event with a snapshot of the ready queue; the step-by-step player shows these events, and a test checks that replaying them rebuilds the Gantt chart.

2 Worked examples: Workloads A and B

I calculated all eight timelines and metrics by hand on paper before checking them with the simulator (scans in Appendix A). The values were transcribed into `tests/oracle/hand.json`; an automated test (`tests/oracle.test.js`) checks the arithmetic of the hand values and compares them with the simulator. In each example below, the upper Gantt row is the hand timeline (from `hand.json`) and the lower row is the simulator's output; in the tables, H = hand and S = simulator (a differing simulator value would be shown in red). Start = first start time, Compl. = completion, TAT = turnaround, WT = waiting, RT = response.

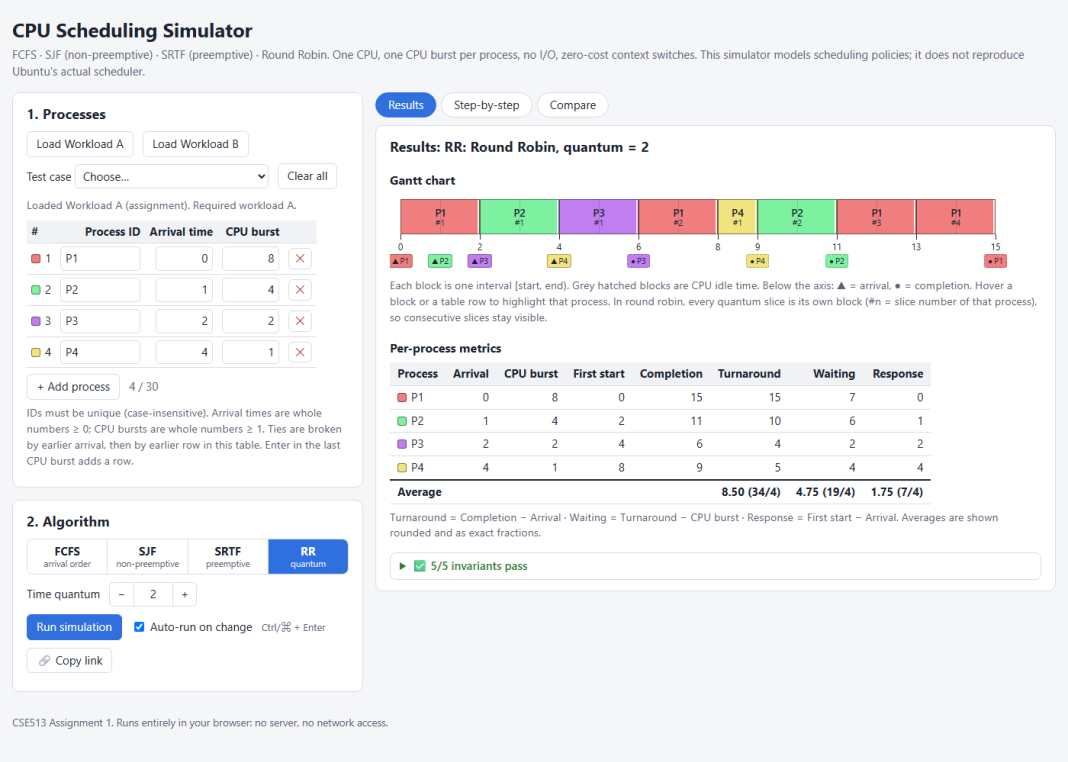


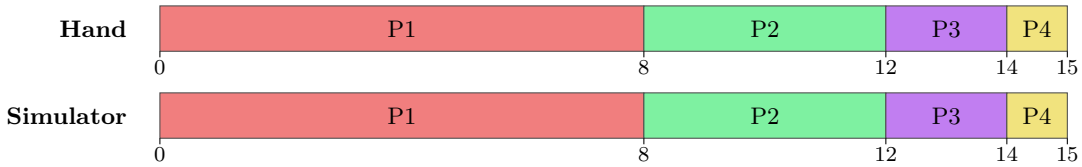
Figure 1: The website with Workload A, RR $q = 2$: input on the left; Gantt chart with arrival (▲) and completion (●) markers, results table and verification badge on the right.

2.1 Workload A

Process	Arrival	CPU burst
P1	0	8
P2	1	4
P3	2	2
P4	4	1

A1. FCFS

At $t = 0$ only P1 is ready and runs to completion at 8 (non-preemptive). P2, P3 and P4 then run in arrival order. The long first job delays every later one (convoy effect).

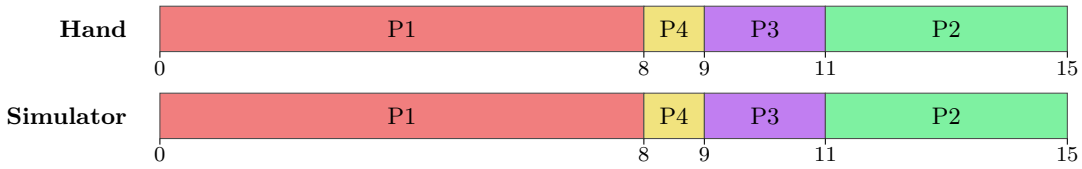


			Start		Compl.		TAT		WT		RT	
Proc.	Arr.	Burst	H	S	H	S	H	S	H	S	H	S
P1	0	8	0	0	8	8	8	8	0	0	0	0
P2	1	4	8	8	12	12	11	11	7	7	7	7
P3	2	2	12	12	14	14	12	12	10	10	10	10
P4	4	1	14	14	15	15	11	11	10	10	10	10
Average (exact fractions)							42/4		27/4		27/4	

Comparison. Hand (H) and simulator (S) agree on all 4 Gantt segments, all 20 per-process values and the 3 averages (turnaround 10.50 (42/4), waiting 6.75 (27/4), response 6.75 (27/4)). No discrepancy. The simulator's 5 invariants: 5/5 pass.

A2. SJF (non-preemptive)

P1 is alone at $t = 0$ and keeps the CPU until 8. At 8 the ready bursts are P2 = 4, P3 = 2, P4 = 1, so P4 runs, then P3 ($2 < 4$), then P2.

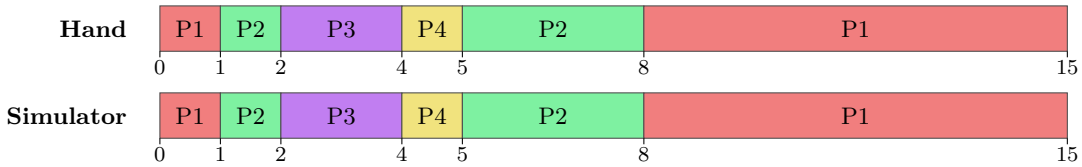


			Start		Compl.		TAT		WT		RT	
Proc.	Arr.	Burst	H	S	H	S	H	S	H	S	H	S
P1	0	8	0	0	8	8	8	8	0	0	0	0
P2	1	4	11	11	15	15	14	14	10	10	10	10
P3	2	2	9	9	11	11	9	9	7	7	7	7
P4	4	1	8	8	9	9	5	5	4	4	4	4
Average (exact fractions)							36/4		21/4		21/4	

Comparison. Hand (H) and simulator (S) agree on all 4 Gantt segments, all 20 per-process values and the 3 averages (turnaround 9.00 (36/4), waiting 5.25 (21/4), response 5.25 (21/4)). No discrepancy. The simulator's 5 invariants: 5/5 pass.

A3. SRTF (preemptive)

$t = 1$: P2 arrives with $4 <$ P1's remaining 7, so P1 is preempted. $t = 2$: P3 (2) $<$ P2's remaining 3, so P2 is preempted. $t = 4$: P3 completes and P4 arrives; P4 (1) is the shortest of P4, P2 (3), P1 (7). P2 then finishes at 8 and P1 at 15. Every process starts the moment it arrives.

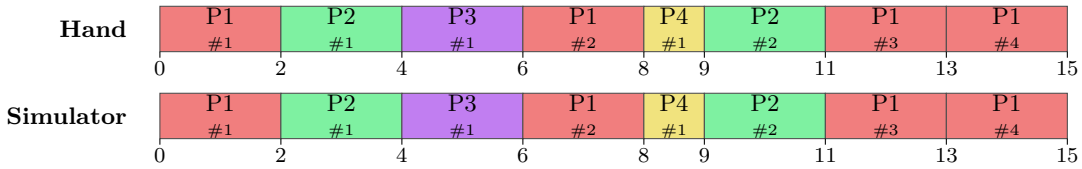


			Start		Compl.		TAT		WT		RT	
Proc.	Arr.	Burst	H	S	H	S	H	S	H	S	H	S
P1	0	8	0	0	15	15	15	15	7	7	0	0
P2	1	4	1	1	8	8	7	7	3	3	0	0
P3	2	2	2	2	4	4	2	2	0	0	0	0
P4	4	1	4	4	5	5	1	1	0	0	0	0
Average (exact fractions)							25/4		10/4		0/4	

Comparison. Hand (H) and simulator (S) agree on all 6 Gantt segments, all 20 per-process values and the 3 averages (turnaround 6.25 (25/4), waiting 2.50 (10/4), response 0.00 (0/4)). No discrepancy. The simulator's 5 invariants: 5/5 pass.

A4. Round Robin, $q = 2$

P1 runs $[0, 2)$. At $t = 2$, P3 arrives exactly at the boundary, so the queue is P2, P3, P1 (P3 before the preempted P1). At $t = 4$, P4 arrives at P2's boundary: queue P3, P1, P4, P2. P3 and P4 finish inside their slices; P1 needs four slices, and the last two are consecutive (#3, #4) because it is then alone. The chart keeps them as two blocks.



			Start		Compl.		TAT		WT		RT	
Proc.	Arr.	Burst	H	S	H	S	H	S	H	S	H	S
P1	0	8	0	0	15	15	15	15	7	7	0	0
P2	1	4	2	2	11	11	10	10	6	6	1	1
P3	2	2	4	4	6	6	4	4	2	2	2	2
P4	4	1	8	8	9	9	5	5	4	4	4	4
Average (exact fractions)							34/4		19/4		7/4	

Comparison. Hand (H) and simulator (S) agree on all 8 Gantt segments, all 20 per-process values and the 3 averages (turnaround 8.50 (34/4), waiting 4.75 (19/4), response 1.75 (7/4)). No discrepancy. The simulator's 5 invariants: 5/5 pass.

2.2 Workload B

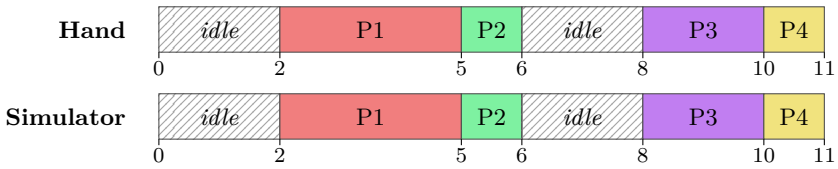
Process	Arrival	CPU burst
P1	2	3
P2	2	1
P3	8	2
P4	9	1

(My handwritten page for B is titled "Workload A" by mistake; its process table and all values are

Workload B's.)

B1. FCFS

The CPU is idle until the first arrival at $t = 2$. P1 and P2 arrive together, so the tie is broken by input order: P1 first. The CPU idles again in $[6, 8)$; P4 arrives at 9 while P3 runs and waits until 10.

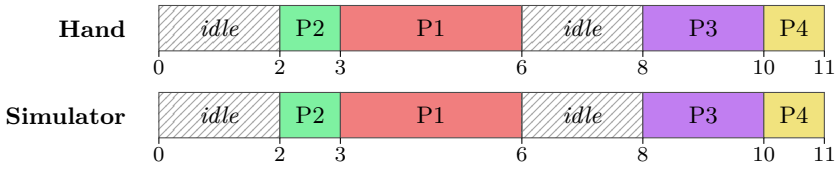


			Start		Compl.		TAT		WT		RT	
Proc.	Arr.	Burst	H	S	H	S	H	S	H	S	H	S
P1	2	3	2	2	5	5	3	3	0	0	0	0
P2	2	1	5	5	6	6	4	4	3	3	3	3
P3	8	2	8	8	10	10	2	2	0	0	0	0
P4	9	1	10	10	11	11	2	2	1	1	1	1
Average (exact fractions)							11/4		4/4		4/4	

Comparison. Hand (H) and simulator (S) agree on all 6 Gantt segments, all 20 per-process values and the 3 averages (turnaround 2.75 (11/4), waiting 1.00 (4/4), response 1.00 (4/4)). No discrepancy. The simulator's 5 invariants: 5/5 pass.

B2. SJF (non-preemptive)

At $t = 2$ P2 (burst 1) is shorter than P1 (3) and runs first. At 9, P4 arrives while P3 runs; SJF does not preempt, so P4 waits until 10.



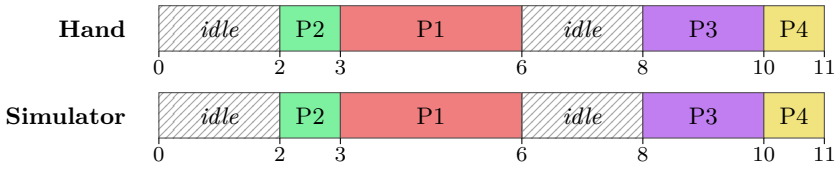
			Start		Compl.		TAT		WT		RT	
Proc.	Arr.	Burst	H	S	H	S	H	S	H	S	H	S
P1	2	3	3	3	6	6	4	4	1	1	1	1
P2	2	1	2	2	3	3	1	1	0	0	0	0
P3	8	2	8	8	10	10	2	2	0	0	0	0
P4	9	1	10	10	11	11	2	2	1	1	1	1
Average (exact fractions)							9/4		2/4		2/4	

Comparison. Hand (H) and simulator (S) agree on all 6 Gantt segments, all 20 per-process values and the 3 averages (turnaround 2.25 (9/4), waiting 0.50 (2/4), response 0.50 (2/4)). No discrepancy. The simulator's 5 invariants: 5/5 pass.

B3. SRTF (preemptive)

Same decisions as SJF. The interesting moment is $t = 9$: P4 arrives with remaining time 1, equal to P3's remaining 1. Preemption needs a *strictly* shorter time, so P3 keeps the CPU (the event log

shows a “keep” event).

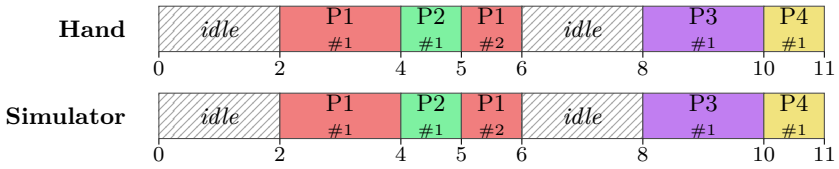


			Start		Compl.		TAT		WT		RT	
Proc.	Arr.	Burst	H	S	H	S	H	S	H	S	H	S
P1	2	3	3	3	6	6	4	4	1	1	1	1
P2	2	1	2	2	3	3	1	1	0	0	0	0
P3	8	2	8	8	10	10	2	2	0	0	0	0
P4	9	1	10	10	11	11	2	2	1	1	1	1
Average (exact fractions)							9/4		2/4		2/4	

Comparison. Hand (H) and simulator (S) agree on all 6 Gantt segments, all 20 per-process values and the 3 averages (turnaround 2.25 (9/4), waiting 0.50 (2/4), response 0.50 (2/4)). No discrepancy. The simulator’s 5 invariants: 5/5 pass.

B4. Round Robin, $q = 2$

P1 (earlier row) runs $[2, 4)$; at its boundary it rejoins behind P2. P2 needs only 1 unit and releases the CPU at 5 (mid-quantum, so P1 gets a fresh slice at 5). P3 runs $[8, 10)$; P4 arrives at 9 and joins the queue. P3 finishes exactly at its boundary, which is a completion, not a quantum expiry, so P3 does not rejoin the queue.



			Start		Compl.		TAT		WT		RT	
Proc.	Arr.	Burst	H	S	H	S	H	S	H	S	H	S
P1	2	3	2	2	6	6	4	4	1	1	0	0
P2	2	1	4	4	5	5	3	3	2	2	2	2
P3	8	2	8	8	10	10	2	2	0	0	0	0
P4	9	1	10	10	11	11	2	2	1	1	1	1
Average (exact fractions)							11/4		4/4		3/4	

Comparison. Hand (H) and simulator (S) agree on all 7 Gantt segments, all 20 per-process values and the 3 averages (turnaround 2.75 (11/4), waiting 1.00 (4/4), response 0.75 (3/4)). No discrepancy. The simulator’s 5 invariants: 5/5 pass.

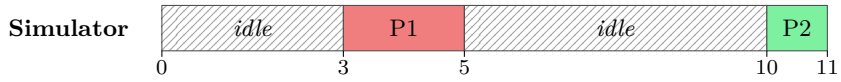
Discrepancies. None: the simulator and my hand calculations agree on every segment and every value in all eight cases. The only issues on the paper were labeling slips that did not affect any value (the B page heading, and in test case T2 a block labeled “P2” that is P3 by its color and by the table; see Section 3).

3 Own test cases

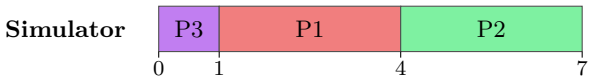
Seven own workloads are built into the website ("Test case" menu). They are checked by the rule tests in `tests/rules.test.js`. The expected schedules were written from the rule text before the engine existed. T1–T3 are the three required cases; for those I also did the full hand calculation (Appendix A), and the oracle test compares those hand values with the simulator too.

Test	Behavior checked	Input (arrival, burst)	Expected Gantt (spec)	Actual
T1	CPU idle at the start and in the middle.	P1(3,2) P2(10,1), FCFS	`idle[0,3) P1[3,5) idle[5,10) P2[10,11)`	✓
T2	Simultaneous arrivals; equal bursts are broken by input order.	P1(0,3) P2(0,3) P3(0,1), SJF	`P3[0,1) P1[1,4) P2[4,7)`	✓
T3	A strictly shorter arrival preempts the running process.	P1(0,6) P2(2,2), SRTF	`P1[0,2) P2[2,4) P1[4,8)`	✓
T4	An arrival exactly at a quantum boundary is enqueued before the preempted process.	P1(0,3) P2(2,2), RR $q = 2$	`P1[0,2) P2[2,4) P1[4,5)`	✓
T5	Equal remaining time does not preempt.	P1(0,3) P2(2,1), SRTF	`P1[0,3) P2[3,4)`	✓
T6	A process finishing mid-quantum releases the CPU and never re-enters the queue.	P1(0,1) P2(0,2), RR $q = 2$	`P1[0,1) P2[1,3)`	✓
T7	Consecutive slices of the same process stay separate and visible.	P1(0,5), RR $q = 2$	`P1[0,2) P1[2,4) P1[4,5)`	✓

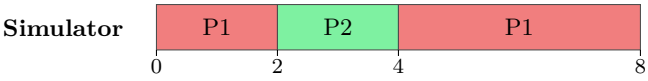
T1: CPU idle time (FCFS). P1(3,2), P2(10,1). Expected: idle $[0, 3)$, P1 $[3, 5)$, idle $[5, 10)$, P2 $[10, 11)$; both processes start on arrival, so TAT = 2 and 1, WT = RT = 0. This checks the leading idle interval and an idle gap between two processes.



T2: simultaneous arrivals and a tie (SJF). P1(0,3), P2(0,3), P3(0,1), all at $t = 0$. Expected: P3 $[0, 1)$ (shortest), then P1 $[1, 4)$ before P2 $[4, 7)$: P1 and P2 tie on burst and on arrival, so the earlier input row wins. WT = 1, 4, 0 (average 5/3). The event text for P1's dispatch states the reason ("tie on burst and arrival with P2 → earlier input row wins").



T3: SRTF preempts a running process. P1(0,6), P2(2,2). Expected: P1 $[0, 2)$; at $t = 2$ P2's 2 < P1's remaining 4, so P1 is preempted; P2 $[2, 4)$; P1 $[4, 8)$. WT(P1) = 2, WT(P2) = 0, RT = 0 for both.



T4–T7 check the remaining rules: an RR arrival exactly at a quantum boundary goes before the preempted process (T4); an equal remaining time does not preempt in SRTF (T5); a process finishing mid-quantum releases the CPU and never re-enters the queue (T6); consecutive slices of one process stay separate blocks (T7). Each test asserts the segments and, where relevant, the order of events at the critical time (for T4: arrive, quantum-expire, dispatch, with the queue P2, P1).

4 Verification

Five invariants. `src/verify.js` checks the required properties on every run, both in the website (verification badge) and in the tests. It recomputes everything from the Gantt segments and the input only, so it does not trust the engine's own numbers:

1. every process receives exactly its CPU burst (sum of its segment lengths);
2. no process runs before its arrival;
3. processes do not overlap on the single CPU (segments are ordered, positive-length and contiguous from $t = 0$);
4. waiting and response times are nonnegative (and the results table matches the Gantt chart);
5. the CPU is never idle while a process is ready.

`tests/invariants.test.js` runs them on all built-in workloads with every algorithm, and feeds deliberately broken schedules (overlap, early start, missing work, idle while ready, ...) to check that each invariant actually detects its violation.

Test suite. `node --test` runs 94 tests: 90 pass, 0 fail, 4 skipped (the oracle slots for T4–T7, which have no hand calculation; a missing hand value is reported as skipped, never as passed).

- *Rule tests*: T1–T7 plus edge cases (many simultaneous arrivals, a single process, burst 1 everywhere, RR with a huge quantum equals FCFS, input order different from ID order, an RR process finishing exactly at a boundary).
- *Validation tests*: every invalid-input case and its message.
- *Oracle*: the 8 Workload A/B cases and T1–T3 against my hand calculations (all match).
- *Property test*: 2,000 random workloads (seeded, so reproducible) × FCFS, SJF, SRTF and RR with $q = 1 \ldots 5$. Every schedule must satisfy the five invariants and the event replay. Also checked: SRTF never has a higher average waiting time than any other algorithm, RR with $q \geq$ the longest burst equals FCFS, all algorithms have the same makespan, and SJF equals SRTF when all processes arrive together.
- *Build test*: the committed `index.html` equals a fresh build, and its bundled engine gives the same results as the source modules.

Do the tests catch real mistakes? We deliberately broke copies of the engine (mutation testing) and ran the suite against each copy:

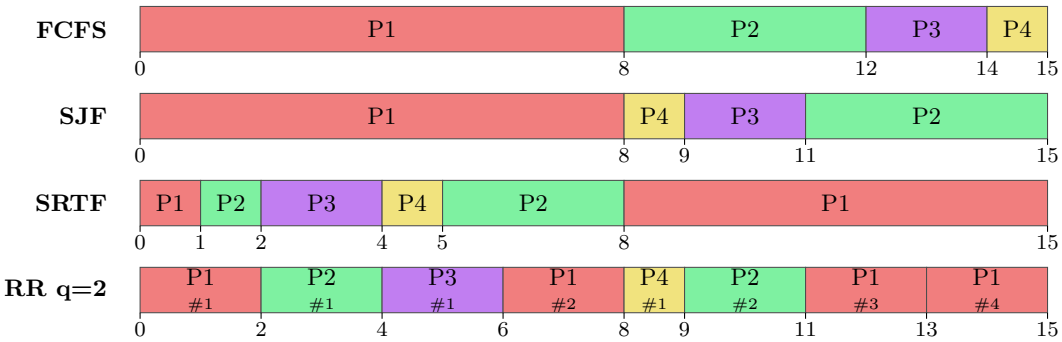
Mutant	Deliberate mistake	Failing tests
A	SRTF preempts on equal remaining time ($\leq$ instead of $<$)	1
B	RR re-enqueues the expired process before same-time arrivals	7
C	Ties broken by process ID instead of input row	2
D	CPU may idle while a process is ready	31
E	SJF preempts like SRTF	1
F	Completion step disabled	loop guard error

All six are caught. A useful observation: the invariant and property tests *alone* miss A and E, because those mutants still produce *valid* schedules; they just follow the wrong policy. Validity and policy compliance are different things, so the rule tests and the hand-calculated oracle are needed in addition to the invariants.

5 Analysis

Q1. Workload A: lowest average waiting and response time

Algorithm	Avg turnaround	Avg waiting	Avg response	Switches
FCFS	10.50 (42/4)	6.75 (27/4)	6.75 (27/4)	3
SJF	9.00 (36/4)	5.25 (21/4)	5.25 (21/4)	3
SRTF	**6.25 (25/4)**	**2.50 (10/4)**	**0.00 (0/4)**	5
RR ($q = 2$)	8.50 (34/4)	4.75 (19/4)	1.75 (7/4)	6



SRTF gives both the lowest average waiting time, 2.50 (10/4), and the lowest average response time, 0.00.

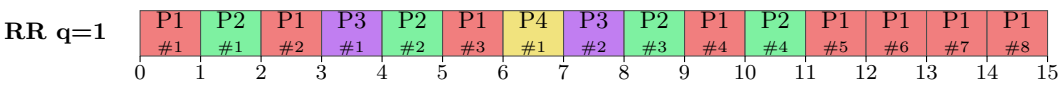
Waiting. Workload A has one long job (P1, 8 units) that arrives first, followed by three short ones. FCFS and SJF are non-preemptive, so once P1 starts at $t = 0$ every later process waits for it to finish at 8 (FCFS 6.75, SJF 5.25; SJF is better only because it orders the three short jobs by length after 8). SRTF interrupts P1 as soon as a shorter job arrives (P2 at 1, then P3 at 2, then P4 at 4), so the short jobs finish early and only P1 waits for long (7 of the total 10 units of waiting). Running the shortest remaining job first is optimal for total waiting time when context switches are free (shortest-remaining-processing-time rule); the property test found no workload where another algorithm beats SRTF on average waiting.

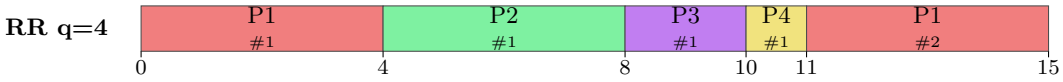
Response. In SRTF every process of Workload A is dispatched at its arrival: each new arrival is shorter than the remaining time of the process running then, so response time is 0 for all four. RR is second (1.75): a new arrival waits at most one quantum for each process queued ahead of it. FCFS and SJF have the worst response time because P2–P4 must wait for P1. RR also has the most process switches (6), which would cost time on a real CPU.

Q2. Round robin with $q = 1, 2, 4$

Workload	q	Avg turnaround	Avg waiting	Avg response	Slices	Switches
A	1	8.50 (34/4)	4.75 (19/4)	0.75 (3/4)	15	11
	2	8.50 (34/4)	4.75 (19/4)	1.75 (7/4)	8	6
	4	9.25 (37/4)	5.50 (22/4)	3.75 (15/4)	5	4
B	1	2.50 (10/4)	0.75 (3/4)	0.25 (1/4)	7	5
	2	2.75 (11/4)	1.00 (4/4)	0.75 (3/4)	5	4
	4	2.75 (11/4)	1.00 (4/4)	1.00 (4/4)	4	3

(Slices = Gantt blocks; switches = changes from one process to a different one.)





- *Response time* improves steadily as q shrinks (A: $3.75 \to 1.75 \to 0.75$), because a newly arrived process waits for at most one short slice of each process ahead of it.
- *Turnaround and waiting* do not follow. In A they are *identical* for $q = 1$ and $q = 2$ (8.50 and 4.75); only the distribution changes (with $q = 1$, P3 waits 4 and P4 waits 2; with $q = 2$ it is the reverse). In B, $q = 2$ and $q = 4$ give the same turnaround and waiting. $q = 4$ is worse in A because it lets P1 and P2 run 4 units each before the short jobs P3 and P4 get the CPU, so RR moves towards FCFS (with $q \geq$ the longest burst it *is* FCFS, which the property test checks).
- *Overhead* grows fast: in A, $q = 1$ needs 15 slices and 11 switches against 8 and 6 for $q = 2$. Here a switch is free, but on a real system each switch costs time, so the same schedule would get slower.

A smaller quantum can even make turnaround *worse*. With two equal jobs P1(0,2), P2(0,2):

- $q = 1$: `P1[0,1) P2[1,2) P1[2,3) P2[3,4)`, average turnaround 3.50 (7/2), average response 0.50 (1/2).
- $q = 2$: `P1[0,2) P2[2,4)`, average turnaround 3.00 (6/2), average response 1.00 (2/2).

With $q = 1$ the jobs interleave and *both* finish late (3 and 4), instead of one finishing at 2. Time slicing helps short jobs reach the CPU sooner; it does not help jobs that are similar in length, because it delays everyone's completion.

Q3. When do SJF and SRTF produce the same schedule?

SRTF behaves differently from SJF only when it preempts, i.e. when a process arrives with a burst *strictly shorter* than the remaining time of the running process. If that never happens, every SRTF decision is made when the CPU becomes free, exactly as in SJF, with the same tie rule, so the schedules are identical. Typical cases:

- all processes arrive at the same time (nothing arrives later). The property test checks this on 2,000 random workloads; T2 is an example;
- processes arrive only while the CPU is idle, or every later arrival is at least as long as the remaining time of the running process.

Example: Workload B. SJF and SRTF give the same Gantt chart (Sections B2 and B3) and the same averages. The only candidate for a preemption is $t = 9$, where P4's 1 equals P3's remaining 1; an equal time does not preempt, so SRTF keeps P3 running like SJF. If P4's burst were shorter than P3's remaining time at its arrival (e.g. P3 burst 3, so remaining 2 at $t = 9$, and P4 burst 1), SRTF would preempt and the schedules would differ. Workload A is a counterexample: there SRTF preempts three times.

Q4. Why must a real OS estimate CPU bursts?

In this simulator the CPU burst is part of the input, so SJF and SRTF can use the exact value. A real operating system does not know how long a process will compute before it blocks or exits: that depends on the program's code, its input data, I/O and user interaction, so it can only be known after the fact. Real processes also alternate between many CPU bursts and I/O, unlike the single burst modeled here. A scheduler that wants SJF-like behavior must therefore *predict* the next burst from past behavior, typically with an exponential average $\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n$, where t_n is the

length of the last burst and τ_n the previous estimate. A wrong estimate gives a worse schedule than the ideal one computed here. This is one reason why general-purpose schedulers (such as Linux's) are not pure SJF/SRTF and instead use time-sharing and fairness ideas; this simulator models the textbook policies only, not Ubuntu's scheduler.

Q5. Can starvation occur under SJF or SRTF?

Yes. Both always prefer the shortest (remaining) job, so a long job can be postponed indefinitely if shorter jobs keep arriving. Two examples, with P1 of burst 10 arriving at $t = 0$:

- *SRTF*: a new job of burst 1 arrives at every time unit $t = 1, 2, 3, \ldots$ P1 runs $[0, 1)$, then every new job preempts it ($1 < 9$) and finishes just as the next one arrives, so P1 never runs again.
- *SJF*: a job of burst 2 also arrives at $t = 0$ (so it starts first), and another burst-2 job arrives at every time unit. Whenever the CPU becomes free, a shorter job than P1 is ready, so P1 never starts.

In both cases P1 waits as long as the stream continues.

In the finite workloads used here this cannot last: there is a last arrival, after which no new job can overtake anyone, and the scheduler never idles while work is waiting. So every process completes by (last arrival time + total CPU burst); the engine even uses this bound as a safety check. A long job can be delayed a lot (in Workload A, P1 waits 7 units under SRTF), but not forever. Real systems with an unbounded stream of jobs need countermeasures such as *aging* (raising the priority of a process the longer it waits), or a time-sharing policy like RR.

Q6. An implementation mistake and how a test exposed it

The engine's main loop originally had no termination guard: it ran "until every process is finished". During mutation testing (Section 4), a mutant with the completion step disabled made the test run hang for more than 10 minutes instead of failing, because no process was ever marked finished and the loop never ended. A test did not fail; it never stopped, and that exposed the mistake. In the browser the same kind of bug would freeze the page without any message. The fix is a horizon guard: since the engine never idles while work is waiting, a correct run always finishes by (last arrival + total burst); past that time `simulate` throws "Internal error: simulation passed t=... without finishing". The mutant now fails immediately with that message, and the property tests (11 random seeds) confirm that the guard never fires on a correct run.

A second example from the website: in the comparison table, each average cell showed "[object HTMLTableCellElement]" instead of a number. An automated browser check (a headless Chrome screenshot of a test workload) exposed it: the DOM helper flattened nested child lists only one level deep. After the fix the check showed the numbers.

6 AI-use statement

Tools. I used Claude Code (Anthropic's coding agent, Claude Opus model) for: analyzing the assignment and drafting a first plan (I then rewrote it into the final plan and its rules); implementing the engine, validation, verification, build script and user interface; writing the tests; debugging; publishing to GitHub Pages; transcribing my handwritten solution pages into `hand.json`; generating the tables and charts of this report; and drafting its text. I did the hand calculations myself on paper, reviewed every phase at a checkpoint, and reviewed and edited this report.

How I checked the output.

- My hand calculations are the independent reference: the oracle test compares them with the

simulator, and it also checks my own arithmetic. A missing hand value is reported as skipped, never passed.

- The five invariants are recomputed from the Gantt chart alone, on every run and in the tests; randomized property tests (2,000 seeded workloads) cover cases nobody wrote by hand.
- Mutation testing showed that the tests detect deliberately broken engines (Section 4).
- The website was checked with automated browser runs and by me, including comparing my paper results with it.
- A development log (`docs/dev-log.md`) records each decision, bug and rejected suggestion at the time it happened; this statement and Q6 are based on it.

Suggestions corrected or rejected.

- In the planning step the AI calculated the Workload A/B results itself and printed them. I rejected this: the hand calculations must be mine and independent, otherwise the oracle would compare the AI with itself. The file was deleted, and the AI was not allowed to write expected values from then on.
- The AI reported which tests a mutant broke before it had actually verified the test names. When I asked, it reran the check and corrected the record.
- When I asked it to fill in the test-case hand values because "the results are the same as the website", it declined to copy the simulator's output into the hand-calculation file (that would make the check circular). I added my handwritten pages instead, and they were transcribed.

I remain responsible for the submitted work and can explain the implementation and trace its decisions for a new workload.

7 Repository and submission

- Repository: `https://github.com/Meliksahcaliskan/Scheduler_Simulation`
- Website: `https://meliksahcaliskan.github.io/Scheduler_Simulation/`
- The source ZIP contains the runnable `index.html`, the sources (`src/`), tests, build and report scripts, `README.md`, the development log and my handwritten solutions.
- Reproduce: `node --test` runs the tests; `node scripts/build.mjs` rebuilds the page; `node scripts/report-data.mjs` regenerates this report's tables and charts.

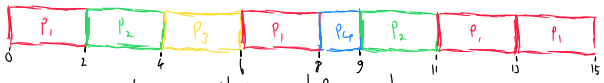
A Handwritten calculations

My paper calculations, as handed in to `tests/oracle/`. Workloads A and B, and test cases 1–3 with all four algorithms (only the algorithm each test case is about is compared automatically).

Workload A p = 2

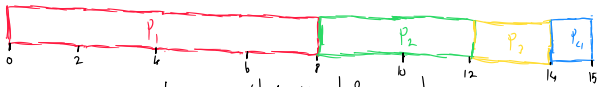
Process	Arrival time	CPU Burst
P_1	0	8
P_2	1	4
P_3	2	2
P_4	4	1

Round Robin:



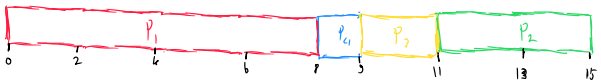
	Turnaround	Waiting	Response
P_1	15 - 0 = 15	15 - 8 = 7	0 - 0 = 0
P_2	11 - 1 = 10	10 - 4 = 6	2 - 1 = 1
P_3	6 - 2 = 4	4 - 2 = 2	4 - 2 = 2
P_4	9 - 4 = 5	5 - 1 = 4	8 - 4 = 4

First Come First Serve:



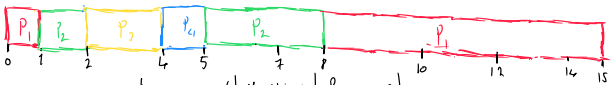
	Turnaround	Waiting	Response
P_1	8 - 0 = 8	8 - 8 = 0	0 - 0 = 0
P_2	12 - 1 = 11	11 - 4 = 7	8 - 1 = 7
P_3	14 - 2 = 12	12 - 2 = 10	12 - 2 = 10
P_4	15 - 4 = 11	11 - 1 = 10	14 - 4 = 10

Shortest Job First:



	Turnaround	Waiting	Response
P_1	8 - 0 = 8	8 - 8 = 0	0 - 0 = 0
P_2	15 - 1 = 14	14 - 4 = 10	11 - 1 = 10
P_3	11 - 2 = 9	9 - 2 = 7	9 - 2 = 7
P_4	9 - 4 = 5	5 - 1 = 4	8 - 4 = 4

Shortest Remaining Time First:

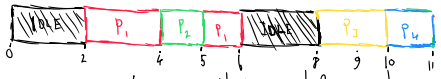


	Turnaround	Waiting	Response
P_1	15 - 0 = 15	15 - 8 = 7	0 - 0 = 0
P_2	8 - 1 = 7	1 - 4 = 3	1 - 1 = 0
P_3	4 - 2 = 2	2 - 2 = 0	2 - 2 = 0
P_4	5 - 4 = 1	1 - 1 = 0	4 - 4 = 0

Workload A p = 2

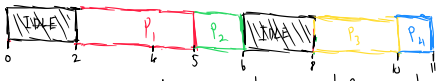
Process	Arrival time	CPU Burst
P_1	2	3
P_2	2	1
P_3	8	2
P_4	9	1

Round Robin:



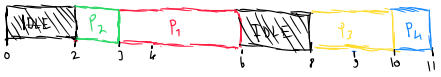
	Turnaround	Waiting	Response
P_1	6 - 2 = 4	4 - 3 = 1	2 - 2 = 0
P_2	5 - 2 = 3	3 - 1 = 2	4 - 2 = 2
P_3	10 - 8 = 2	2 - 2 = 0	8 - 8 = 0
P_4	11 - 9 = 2	2 - 1 = 1	10 - 9 = 1

First Come First Serve:



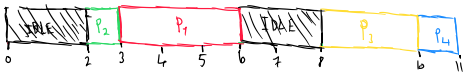
	Turnaround	Waiting	Response
P_1	5 - 2 = 3	3 - 3 = 0	2 - 2 = 0
P_2	6 - 2 = 4	4 - 1 = 3	5 - 2 = 3
P_3	10 - 8 = 2	2 - 2 = 0	8 - 8 = 0
P_4	11 - 9 = 2	2 - 1 = 1	10 - 9 = 1

Shortest Job First:



	Turnaround	Waiting	Response
P_1	6 - 2 = 4	4 - 3 = 1	3 - 2 = 1
P_2	3 - 2 = 1	1 - 1 = 0	2 - 2 = 0
P_3	10 - 8 = 2	2 - 2 = 0	8 - 8 = 0
P_4	11 - 9 = 2	2 - 1 = 1	10 - 9 = 1

Shortest Remaining Time First:



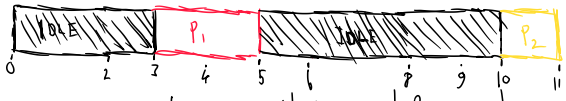
	Turnaround	Waiting	Response
P_1	6 - 2 = 4	4 - 3 = 1	3 - 2 = 1
P_2	3 - 2 = 1	1 - 1 = 0	2 - 2 = 0
P_3	10 - 8 = 2	2 - 2 = 0	8 - 8 = 0
P_4	11 - 9 = 2	2 - 1 = 1	10 - 9 = 1

Test Case 1

Idle Time $P \geq 2$

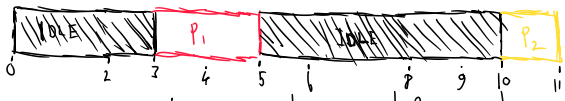
Process	Arrival time	CPU Burst
P_1	3	2
P_2	10	1

Round Robin:



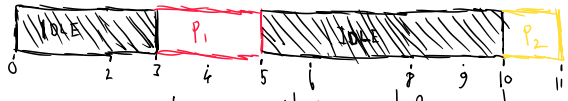
	Turnaround	Waiting	Response
P_1	5 - 3 = 2	2 - 2 = 0	3 - 3 = 0
P_2	11 - 10 = 1	1 - 1 = 0	10 - 10 = 0

First Come First Serve:



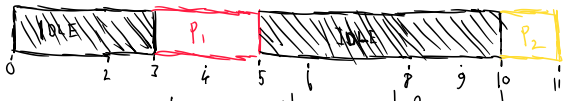
	Turnaround	Waiting	Response
P_1	5 - 3 = 2	2 - 2 = 0	3 - 3 = 0
P_2	11 - 10 = 1	1 - 1 = 0	10 - 10 = 0

Shortest Job First:



	Turnaround	Waiting	Response
P_1	5 - 3 = 2	2 - 2 = 0	3 - 3 = 0
P_2	11 - 10 = 1	1 - 1 = 0	10 - 10 = 0

Shortest Remaining Time First:



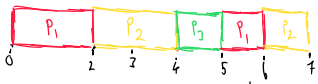
	Turnaround	Waiting	Response
P_1	5 - 3 = 2	2 - 2 = 0	3 - 3 = 0
P_2	11 - 10 = 1	1 - 1 = 0	10 - 10 = 0

Test Case 2 — Simultaneous Arrival + Tie

$P \geq 2$

Process	Arrival time	CPU Burst
P_1	0	3
P_2	0	3
P_3	0	1

Round Robin:



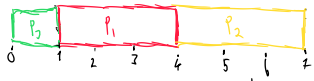
	Turnaround	Waiting	Response
P_1	6 - 0 = 6	6 - 3 = 3	0 - 0 = 0
P_2	7 - 0 = 7	7 - 3 = 4	2 - 0 = 2
P_3	5 - 0 = 5	5 - 1 = 4	4 - 0 = 4

First Come First Serve:



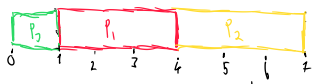
	Turnaround	Waiting	Response
P_1	3 - 0 = 3	3 - 3 = 0	0 - 0 = 0
P_2	6 - 0 = 6	6 - 3 = 3	3 - 0 = 3
P_3	7 - 0 = 7	7 - 1 = 6	6 - 0 = 6

Shortest Job First:



	Turnaround	Waiting	Response
P_1	4 - 0 = 4	4 - 3 = 1	1 - 0 = 1
P_2	7 - 0 = 7	7 - 3 = 4	4 - 0 = 4
P_3	1 - 0 = 1	1 - 1 = 0	1 - 1 = 0

Shortest Remaining Time First:



	Turnaround	Waiting	Response
P_1	4 - 0 = 4	4 - 3 = 1	1 - 0 = 1
P_2	7 - 0 = 7	7 - 3 = 4	4 - 0 = 4
P_3	1 - 0 = 1	1 - 1 = 0	1 - 1 = 0

Test Case 2 SRTF Preempt

P= 2

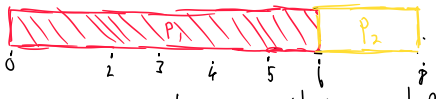
Process	Arrival Time	CPU Burst
P_1	0	6
P_2	2	2

Round Robin:



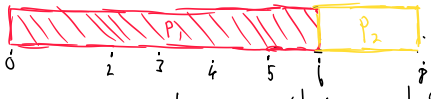
	Turnaround	Waiting	Response
P_1	8-0=8	8-6=2	0-0=0
P_2	4-2=2	2-2=0	2-2=0

First Come First Serve:



	Turnaround	Waiting	Response
P_1	6-0=6	6-6=0	0-0=0
P_2	8-2=6	6-2=4	6-2=4

Shortest Job First:



	Turnaround	Waiting	Response
P_1	6-0=6	6-6=0	0-0=0
P_2	8-2=6	6-2=4	6-2=4

Shortest Remaining Time First:



	Turnaround	Waiting	Response
P_1	8-0=8	8-6=2	0-0=0
P_2	4-2=2	2-2=0	2-2=0